

Data Science

CSE558

Recap

- Universal Hashing

Overview

- Universal Hashing
- Bloom Filters

Hashing



Query: Bought ticket?



Linear search: Time consuming. **Can we do better?**

Hash Table

U: Universe (All people who have pre-registered for the event. Typically its large.)

n: number of tickets sold (Set of people who bought tickets. Typically $n < |U|$.)

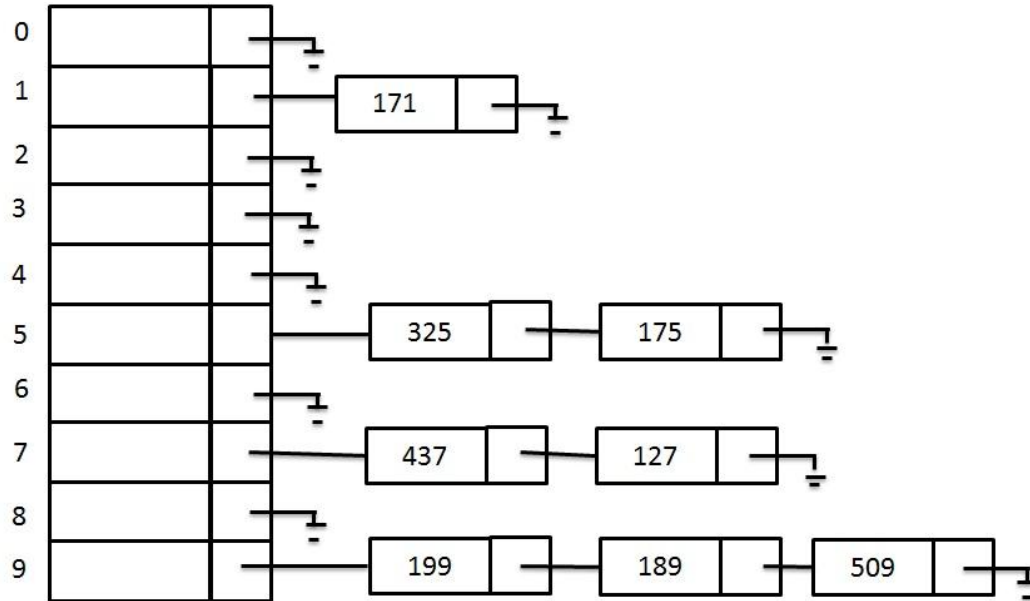
Hash Table

- Array of size m
- Hash function, $h: U \rightarrow \{0, 1, \dots, m-1\}$

Typically, $m \ll |U|$ and $m < n$.

- Collision happens when $x \neq y$ but $h(x) = h(y)$.
- By pigeonhole principle.

Handle Collision



At every index, maintain a chain of all the elements hashed there using a linked list.

What is a good hash function?

Hash function

- Deterministic hash function is not good enough.
 - $h(x) = x \bmod m$.
 - Too many collision if adversary knows the hash function.
 - Leads to linear search in worst case scenario.
- Random hash function
 - $H = \{h_1, h_2, \dots\}$
 - Choose $h \in H$ uniformly at random.
 - Analyze expected query time.
- $|H|$ cannot be too large, e.g., $m^{|U|}$
 - Defeats the purpose quick membership check!

Hash Family

Create a small hash family with good behaviour.

- Universal: $\Pr_{h \in \mathcal{H}} (h(x) = h(y)) = \frac{1}{m}$, for every x and y

Expected Query Time

$\ell(x)$ = length of chain at $h(x)$

Expected query time of $x = O(1 + \mathbf{E}_{h \in \mathcal{H}}[\ell(x)])$

- For both insertion and deletion.

Analyze expected length of chaining.

Consider $(n^2 - n)/2$ random variables as, $C_{xy} = \begin{cases} 1, & \text{if } h(x) == h(y) \\ 0, & \text{otherwise.} \end{cases}$

$$\mathbf{E}[\ell(x)] = \mathbf{E}\left[\sum_y C_{xy}\right] = \sum_y \mathbf{E}[C_{xy}] = \sum_y \mathbf{P}[h(x) == h(y)] = \frac{n-1}{m}$$

Designing Universal Hash Function

For universe U , pick a prime number $p > |U|$.

Define the family $\mathcal{H} = \{h_a(x) = (ax \bmod p) \bmod m, a \in [1, \dots, p-1]\}$

From above family, choosing a hash function $\Rightarrow$ choosing $a \in [1, \dots, p-1]$.

How many $a \in [1, \dots, p-1]$, such that

$$h_a(x) - h_a(y) = (a(x - y) \bmod p) \bmod m = 0$$

At most $(p-1)/m$.

What is the probability of choosing 'a' such that, the above holds, i.e., $a(x-y) \bmod p$ is one of these $(p-1)/m$ numbers (or collision happens)?

Property of Prime

w.l.o.g., $[x - y] = z \in [1, \dots, p-1]$.

For every $t, z \in [1, \dots, p-1]$, there is an unique 'c' $\in [1, \dots, p-1]$ s.t. $cz \bmod p = t$.

We know there are $(p-1)/m$ values of t that will result to a collision.

So,

$$\mathbf{P}(h_a(x) == h_a(y)) = \frac{p-1}{m} \cdot \frac{1}{p-1} = \frac{1}{m}$$

Overview

- Concentration Inequalities
- Universal Hashing
- Bloom Filters

Membership Problem



Check ticket of a
return customer.



Solution

Universe U , such that $n \ll |U|$

Trade off between Time & Space

Can we do better?

- Hash table of size m
 - Space $O(n \log(|U|))$
 - Query time: $O(n/m)$
- Bit Array of size $|U|$
 - Space = $|U|$
 - Query time: $O(1)$

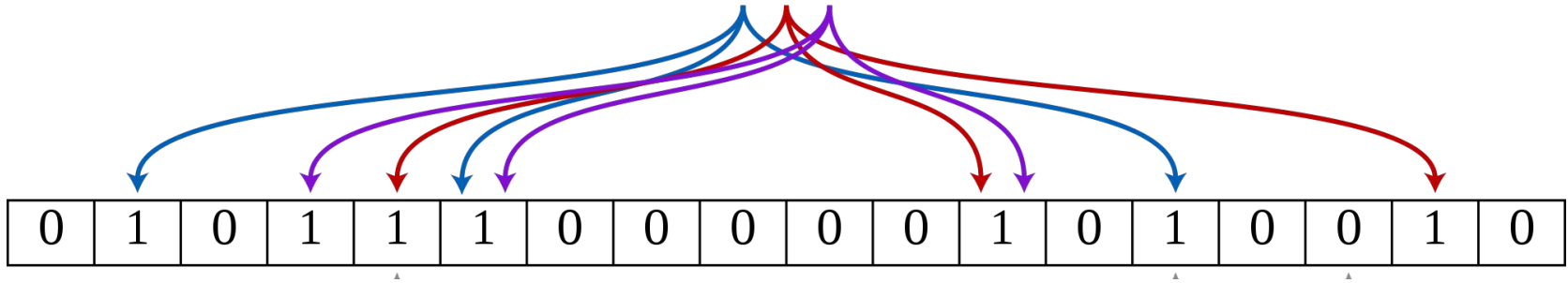
Randomized Algorithm

- Las Vegas algorithm:
 - Randomized algorithm but never returns incorrect answers.
 - Running time is a random variable.
- Monte Carlo algorithm:
 - Randomized algorithm but allows to incorrect answers with low probability.
 - Running time is deterministic.
- Example?

Bloom Filter (Bloom, 1970)

Bit array of size m

k , independent hash functions $h_1, \dots, h_k. h_i: |U| \rightarrow [m]$
 $\{x, y, z\}$



Bloom Filter (Bloom, 1970)

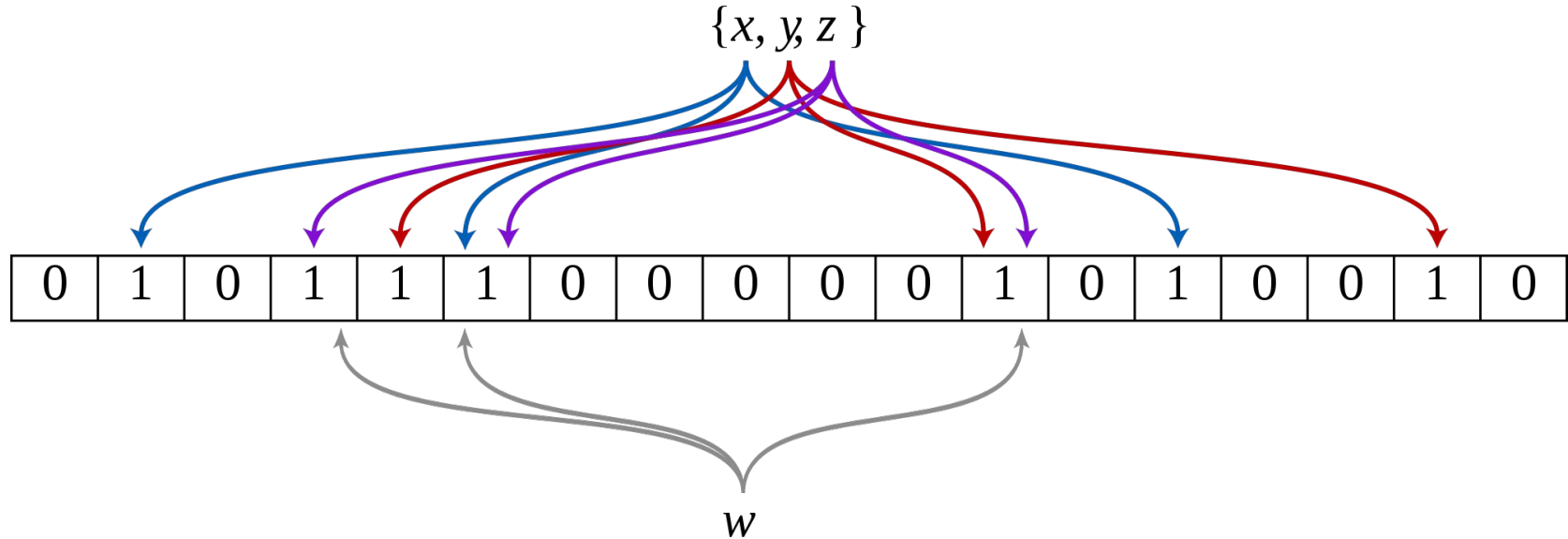
Initialize: $B[i] = 0$ for every $1 \leq i \leq m$

Insert(B, x): $B[h_j(x)] = 1$, for every $1 \leq j \leq k$

Query(B, w): If $B[h_1(w)] \wedge B[h_3(w)] \wedge \dots \wedge B[h_k(w)] = 1$, then return true else false.

Bloom Filter (Bloom, 1970)

If query w is present then it will return true.

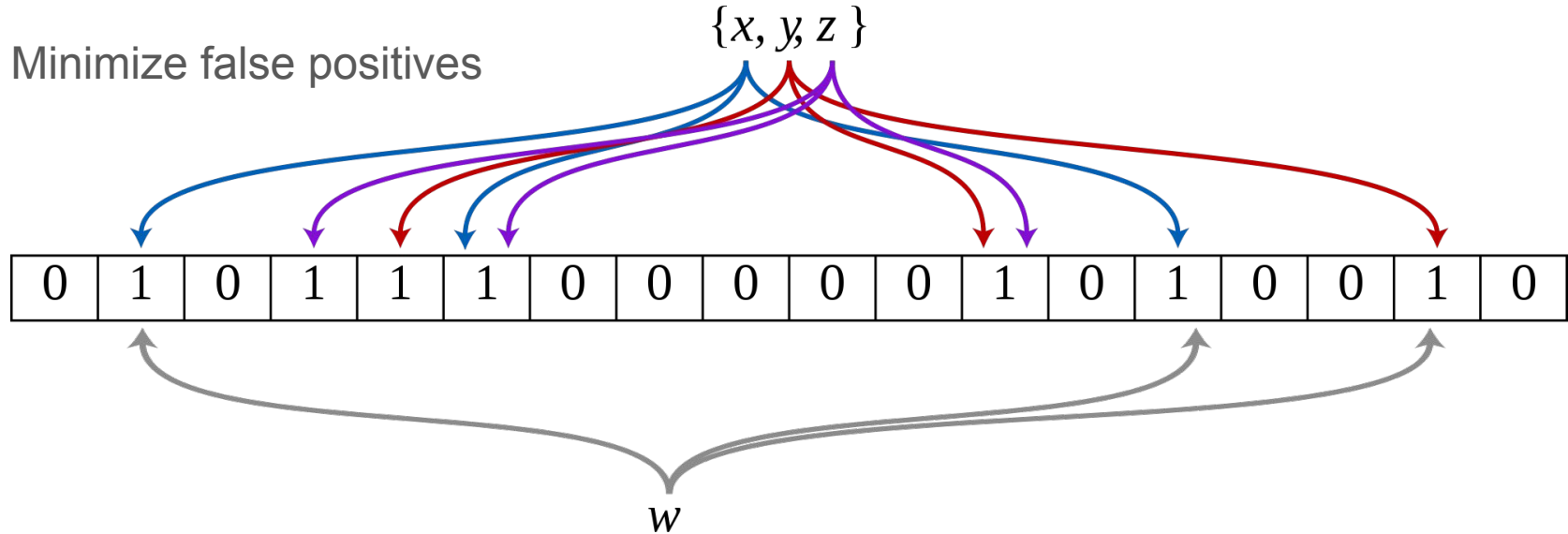


Bloom Filter (Bloom, 1970)

If query w is present then it will return true.

Else when not present then there might be false positive.

Minimize false positives



Bound False Positive

$k = 1$, is a normal bit array

Increasing k helps but needed to computed judiciously.

$|B| = m$, n elements inserted.

If w is not present in the set of n elements, then with what probability $\text{Query}(B, w)$ returns true?

Let $h_1, \dots, h_k$ be k independent hash functions and $P(h_i(\cdot) = j) = 1/m$ for all $1 \leq j \leq m$.

ability that a fixed bit 'j' is **not set**,

$$P(j == 0) = \left(1 - \frac{1}{m}\right)^{kn} \leq \exp\left(\frac{-kn}{m}\right)$$

The expected number of zero bits is at most

Analyze false positives

$|B| = m$, n elements inserted.

If w is not present in the set of n elements, then with what probability $\text{Query}(B, w)$ returns true?

Let $h_1, \dots, h_k$ be k independent hash functions and $P(h_i(\cdot) = j) = 1/m$ for all $1 \leq j \leq m$.

The expected number of zero bits is at most $m \exp\left(\frac{-kn}{m}\right)$. After inserting all n elements, probability that $\text{Query}(B, w)$ returns true is,

$$\mathbf{P}(\text{Query}(B, w) \text{ returns } \textit{True}) = \left(1 - \exp\left(\frac{-kn}{m}\right)\right)^k$$

$$\text{Let, } p = \exp\left(\frac{-kn}{m}\right)$$

Analyze false positives

$|B| = m$, n elements inserted.

If w is not present in the set of n elements, then with what probability $\text{Query}(B, w)$ returns true?

After inserting all n elements, probability that $\text{Query}(B, w)$ returns true is,

$$\mathbf{P}(\text{Query}(B, w) \text{ returns } \textit{True}) = (1 - p)^k$$

$$\log (\mathbf{P}(\text{Query}(B, w) \text{ returns } \textit{True})) = \frac{-m}{n} \ln(p) \ln(1 - p)$$

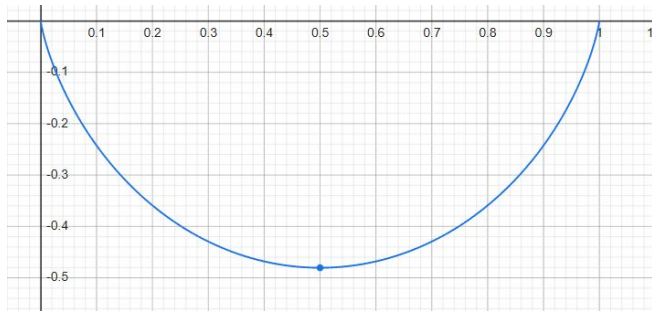
Analyze false positives

$|B| = m$, n elements inserted.

If w is not present in the set of n elements, then with what probability $\text{Query}(B, w)$ returns true?

After inserting all n elements, probability that $\text{Query}(B, w)$ returns true is,

$$\log(\mathbf{P}(\text{Query}(B, w) \text{ returns } \textit{True})) = \frac{-m}{n} \ln(p) \ln(1 - p)$$



Plot $-\ln(p)\ln(1-p)$

Minimizes at $p = 1/2$.

So, $k = (m/n) \cdot \ln(2)$

So, optimal choice of k give false positive

$$2^{-(m/n) \ln(2)}$$

Analyze false positives

$|B| = m$, n elements inserted.

If w is not present in the set of n elements, then with what probability $\text{Query}(B, w)$ returns true?

After inserting all n elements, probability that $\text{Query}(B, w)$ returns true is,

$$\log(\mathbf{P}(\text{Query}(B, w) \text{ returns } \textit{True})) = \frac{-m}{n} \ln(p) \ln(1 - p)$$

Optimal choice of k give false positive rate of $2^{-(m/n) \ln(2)}$

If user wants false positive at most δ then set

$$m = \left\lceil \frac{n \ln(1/\delta)}{\ln(2)} \right\rceil$$